



Wrocław
University
of Science
and Technology

Containers on AWS: ECS Networking, IAM, and Observability — Part 2

Cloud Programming (W04IST-SI0826G)

Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026



HR EXCELLENCE IN RESEARCH





This week

After this class, you should be able to:

- **Design** the VPC and security group architecture for a two-tier ECS deployment
- **Configure** ALB listener rules to route traffic to the correct ECS service
- **Distinguish** the task execution role from the task role and apply least privilege
- **Explain** how container logs flow from `stdout` to CloudWatch Logs
- **Create** a CloudWatch Alarm and describe its notification path via SNS
- **Configure** ECS Service Auto Scaling with a target-tracking policy

Four operational gaps

Last week: a Spring Boot task and a Vite frontend task are running in ECS Fargate.

Four things are still broken:

- 1 **No public traffic** — nobody can reach the Vite frontend
- 2 **Backend exposed** — the Spring Boot service is reachable directly from the internet
- 3 **Blind operations** — CPU has been at 95 % for 20 minutes; nobody noticed
- 4 **Single point of failure** — one task, one crash, zero availability

These four gaps drive the four topic groups in today's lecture.

Quick recall

Without looking at your notes:

- What are the four ECS primitives, from the outside in?
- What is the difference between a task definition and a running task?
- Which launch type did we recommend for new workloads, and why?
- What two IAM roles did we mention in a task definition?

Quick recall

Without looking at your notes:

- What are the four ECS primitives, from the outside in?
- What is the difference between a task definition and a running task?
- Which launch type did we recommend for new workloads, and why?
- What two IAM roles did we mention in a task definition?
- Cluster → Service → Task Definition → Task
- Task definition = blueprint (immutable revision); task = one running instance
- Fargate — no EC2 instances to manage, per-task billing, lower operational overhead
- **Task execution role** (used by the ECS agent) and **task role** (used by your code)

Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer
- 3 IAM Roles for ECS Tasks
- 4 CloudWatch for ECS
- 5 ECS Service Auto Scaling
- 6 Closing



VPC layout for ECS

Two subnet tiers in the same VPC:

- **Public subnets** — Application Load Balancer (ALB)
 - Route table: `0.0.0.0/0 > Internet Gateway`
- **Private subnets** — Vite frontend and Spring Boot backend service
 - Route table: `0.0.0.0/0 > NAT Gateway (outbound only)`

The internet can reach the **ALB only**.

The frontend and backend have **no inbound internet route** — by design.

```
1 Internet
2 |
3 [ALB]      public subnet
4 |  |
5 |  [Vite] private subnet
6 |
7 [Spring] private subnet
```



Security group design

Three security groups — one per boundary:

SG	Inbound	Source
alb-sg	TCP 80, 443	Internet
frontend-sg	TCP 3000	alb-sg
backend-sg	TCP 5000	alb-sg

- **Least privilege**: each SG allows traffic only from the tier above it
- When source is a **security group**, and not a CIDR — security group handles IP changes
- Front- and Backend ports are unreachable from the Internet, even if security groups are misconfigured



Create Security Group I

Security group for the ALB:

```
1 # Security group for the ALB
2 aws ec2 create-security-group \
3   --group-name alb-sg \
4   --description "ALB - public inbound" \
5   --vpc-id vpc-0abc123def456
6
7 aws ec2 authorize-security-group-ingress \
8   --group-id <alb-sg-id> \
9   --protocol tcp --port 80 --cidr 0.0.0.0/0
```

Replace `<alb-sg-id>` with the actual ID from the previous command

Create Security Group II

Security group for the Frontend:

```
1 aws ec2 create-security-group \  
2   --group-name frontend-sg \  
3   --description "Vite frontend - ALB inbound only" \  
4   --vpc-id vpc-0abc123def456  
  
5 # Accept traffic from the ALB SG only (not the internet)  
6 aws ec2 authorize-security-group-ingress \  
7   --group-id <frontend-sg-id> \  
8   --protocol tcp --port 3000 \  
9   --source-group <alb-sg-id>
```

Build security group for backend the similar way

ECS networking mode **awsvpc**

- Each ECS task gets its **own Elastic Network Interface (ENI)**
- Each task has its **own private IP** in the VPC subnet
- Security groups are assigned **directly to the task**
 - not the EC2 instance
- Required for Fargate; recommended for EC2 launch type

Why **awsvpc** matters for security

With **awsvpc**, each task has its own ENI and SG: a compromised task cannot reach other tasks on the same host

Without **awsvpc**, multiple tasks on the same EC2 instance have one common ENI and SG

Before we continue — predict

Your colleague configures the backend security group like this:

```
1 aws ec2 authorize-security-group-ingress \  
2   --group-id <sg-backend-id> \  
3   --protocol tcp --port 8080 \  
4   --cidr 0.0.0.0/0
```

The backend is in a **private subnet**.

- 1 Is the backend now reachable from the internet? Why or why not?
- 2 What is the security risk even if the answer to the first question is “no”?

Before we continue — predict

Your colleague configures the backend security group like this:

```
1 aws ec2 authorize-security-group-ingress \  
2   --group-id <sg-backend-id> \  
3   --protocol tcp --port 8080 \  
4   --cidr 0.0.0.0/0
```

The backend is in a **private subnet**.

- 1 Is the backend now reachable from the internet? Why or why not?
- 2 What is the security risk even if the answer to the first question is “no”?
- 1 Not reachable from Internet
- 2 Reachable by any VPC resource



Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer**
- 3 IAM Roles for ECS Tasks
- 4 CloudWatch for ECS
- 5 ECS Service Auto Scaling
- 6 Closing

Application Load Balancer components

Four building blocks:

- **Load balancer** — the public-facing endpoint; spans multiple AZs
- **Listener** — monitors a port (e.g., TCP 80) and evaluates rules
- **Listener rule** — matches request attributes (path, host, header) and forwards to a target group
- **Target group** — a pool of targets (tasks, IPs, or Lambda functions) that receive traffic

Health checks run continuously: unhealthy targets are removed from routing until they recover



ALB Target Groups for ECS in `awsvpc` mode

- In `awsvpc` mode, target type must be `ip` (not `instance`)
- Each ECS task's ENI IP is registered automatically when the task starts
- ECS deregisters the IP when the task stops (connection draining first)
- Health check: `GET /` for frontend; `GET /api/health` for backend



Create Target Group for ECS in **awsvpc** mode

```
1  # Frontend target group
2  aws elbv2 create-target-group \
3      --name frontend-tg --protocol HTTP --port 3000 \
4      --vpc-id vpc-0abc123def456 \
5      --target-type ip \
6      --health-check-path /

7  # Backend target group
8  aws elbv2 create-target-group \
9      --name backend-tg --protocol HTTP --port 5000 \
10     --vpc-id vpc-0abc123def456 \
11     --target-type ip \
12     --health-check-path /api/health
```

- Compare health check paths



Create ALB listener

```
1 # Create the ALB (internet-facing, public subnets)
2 aws elbv2 create-load-balancer \
3     --name app-alb \
4     --subnets subnet-pub1 subnet-pub2 \
5     --security-groups sg-alb \
6     --type application
```



Create ALB routing rules

We start with the default listener for `/` path (frontend)

```
1 aws elbv2 create-listener \  
2   --load-balancer-arn <app-alb-arn> \  
3   --protocol HTTP --port 80 \  
4   --default-actions Type=forward,TargetGroupArn=<frontend-tg-arn>
```

Then we add a rule for `/api/*` path (backend)

```
1 # higher priority = evaluated first  
2 aws elbv2 create-rule \  
3   --listener-arn <app-alb-listener-arn> \  
4   --conditions Field=path-pattern,Values='/api/*' \  
5   --priority 10 \  
6   --actions Type=forward,TargetGroupArn=<backend-tg-arn>
```

End-to-end request trace

A browser requests `GET /api/users`:

- 1 DNS resolves `app.example.com` to the ALB DNS name, and then **ALB public IP**
- 2 Request arrives on port 80 and triggers **ALB listener**
- 3 Listener evaluates rules: path `/api/users` matches `/api/*` in **rule 10**
- 4 ALB selects a healthy task IP from the **backend target group**
- 5 Request is forwarded to the **Spring Boot task** on port 5000
- 6 Response travels back through the ALB to the browser

A browser requests `GET /`:

- 1 Path does not match `/api/*`, so **default action** is triggered and routes to **frontend target group** and **Vite task** on port 3000



Admin challenge

Your ALB is deployed. The Vite frontend loads correctly at `http://app.example.com/`.

But `http://app.example.com/api/users` returns **HTTP 502 Bad Gateway**.

Name **three** things you would check to solve this issue.

Admin challenge

Your ALB is deployed. The Vite frontend loads correctly at `http://app.example.com/`.

But `http://app.example.com/api/users` returns **HTTP 502 Bad Gateway**.

Name **three** things you would check to solve this issue.

- 1 **ALB listener rule** — does `/api/*` rule exist at priority 10? Is the path pattern correct (with `*`)?
- 2 **Backend target group health** — is at least one target registered and healthy? What does the health check return?
- 3 **Backend SG inbound rule** — does `backend-sg` allow TCP 5000 from `alb-sg`? (in `awsvpc` mode: check task-level SG, not instance SG)



Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer
- 3 IAM Roles for ECS Tasks**
- 4 CloudWatch for ECS
- 5 ECS Service Auto Scaling
- 6 Closing



Two roles — two principals I

	Task Execution Role	Task Role
Who uses it	ECS agent (AWS infrastructure)	Your application code (the process inside the container)
When used	At task launch and while agent manages the task	At runtime, whenever code calls an AWS API
Typical permissions	Pull image from ECR; write logs to CloudWatch; read secrets from Secrets Manager	Read from S3; query DynamoDB; publish to SQS
Required for	Every ECS task	Only tasks that call AWS services



Two roles — two principals II

- Both roles trust `ecs-tasks.amazonaws.com` as the principal
- **They must be separate:** the agent does not need DynamoDB access; the app does not need `ecr:GetAuthorizationToken`

Learner Lab warning

As AWS Learner Lab does not support IAM role creation, both ECS roles must be set to `LabRole` that gives administrative access to all lab resources



Sample Task Execution role

Minimum permissions required by the ECS agent:

```
1  aws iam create-role \  
2    --role-name ecsTaskExecutionRole \  
3    --assume-role-policy-document '{  
4      "Version": "2012-10-17",  
5      "Statement": [{  
6        "Effect": "Allow",  
7        "Principal": { "Service": "ecs-tasks.amazonaws.com" },  
8        "Action": "sts:AssumeRole"  
9      }]  
10   }'  
  
11  aws iam attach-role-policy \  
12    --role-name ecsTaskExecutionRole \  
13    --policy-arn arn:aws:iam::aws:policy/service-role/  
    ↪ AmazonECSTaskExecutionRolePolicy ↵
```

AmazonECSTaskExecutionRolePolicy covers: `ecr:GetAuthorizationToken`,
`ecr:BatchGetImage`, `logs:CreateLogStream`, `logs:PutLogEvents`



Sample Task Role I

First we allow ECS tasks to assume this task role:

```
1 aws iam create-role \  
2   --role-name spring-backend-task-role \  
3   --assume-role-policy-document '{  
4     "Version": "2012-10-17",  
5     "Statement": [{  
6       "Effect": "Allow",  
7       "Principal": { "Service": "ecs-tasks.amazonaws.com" },  
8       "Action": "sts:AssumeRole"  
9     }]  
10  }'
```



Sample Task Role II

Then we allow Spring Boot backend to read from the one table in DynamoDB:

```
1 aws iam put-role-policy \  
2   --role-name spring-backend-task-role \  
3   --policy-name backend-access \  
4   --policy-document '{  
5     "Version": "2012-10-17",  
6     "Statement": [  
7       {  
8         "Effect": "Allow",  
9         "Action": ["dynamodb:GetItem", "dynamodb:Query", ↵  
10        ↵ "dynamodb:Scan"],  
11        "Resource": "arn:aws:dynamodb:us-east-1:123456789012: ↵  
12        ↵ table/Orders"  
13      }  
14    ]  
15  }'
```



Sample Task Role III

Watch the use of **Least Privilege** principle here: only the **necessary** read actions on the **specific** DynamoDB table are allowed.

Discuss with your neighbour (2 min)

Your Spring Boot backend task keeps failing with:

```
1 AccessDeniedException: User: arn:aws:sts::123456789:assumed-role/  
2 spring-backend-task-role/... is not authorized to perform:  
3 dynamodb:GetItem on resource:  
  ↪ arn:aws:dynamodb:us-east-1:123456789012:table/Orders
```



- 1 Which role needs updating — the task execution role or the task role?
- 2 What is the minimum change required?
- 3 Why should you **not** just attach `AmazonDynamoDBFullAccess`?



Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer
- 3 IAM Roles for ECS Tasks
- 4 CloudWatch for ECS**
- 5 ECS Service Auto Scaling
- 6 Closing



What is Amazon CloudWatch?

AWS built-in observability service

Capability	What it stores	Example
Metrics	Numeric time-series data	CPU%, request count, memory
Logs	Arbitrary text from any source	Container stdout, application errors
Alarms	Rules that watch one metric and trigger actions	send email when CPU>80%

- All AWS services emit metrics to CloudWatch **automatically** — no agent needed
- Logs require the application to **send** them explicitly
- Alarms connect metrics to actions: SNS notifications, Auto Scaling policies, Lambda

Container logs in CloudWatch I

- 1 Container writes to stdout/stderr
- 2 `awslogs driver` - configured in task definition
- 3 CloudWatch Logs - log group: `/ecs/<cluster>/<service>`
- 4 Log stream - one stream per task: `/ecs/<container>/<task-id>`

Container logs in CloudWatch II

Task definition log configuration:

```
1 "logConfiguration": {  
2   "logDriver": "awslogs",  
3   "options": {  
4     "awslogs-group":      "/ecs/my-cluster/backend-service",  
5     "awslogs-region":     "us-east-1",  
6     "awslogs-stream-prefix": "ecs"  
7   }  
8 }
```

Prerequisite: task execution role must have `logs:CreateLogStream` and `logs:PutLogEvents`.

Locating logs for a failing container I

Practical workflow:

- 1 Task fails to start
- 2 Find the stream
- 3 Read the error
- 4 Fix the task definition

Locating logs for a failing container II

```
1 # Create the log group (must exist before the first task starts)
2 aws logs create-log-group \
3   --log-group-name /ecs/my-cluster/backend-service

4 # List log streams – most recent first
5 aws logs describe-log-streams \
6   --log-group-name /ecs/my-cluster/backend-service \
7   --order-by LastEventTime --descending \
8   --max-items 5

9 # Read log events from the failing task's stream
10 aws logs get-log-events \
11   --log-group-name /ecs/my-cluster/backend-service \
12   --log-stream-name ecs/backend/abc123def456 \
13   --limit 50
```

Key ECS metrics in CloudWatch

Metric	Namespace	What it measures
CPUUtilization	AWS/ECS	Average CPU used vs. task CPU reservation (%)
MemoryUtilization	AWS/ECS	Average memory used vs. task memory limit (%)
RunningTaskCount	AWS/ECS	Number of running tasks in the service
RequestCountPerTarget	AWS/ApplicationELB	ALB requests per target per minute
TargetResponseTime	AWS/ApplicationELB	P50/P99 latency for target group responses

Dimensions: ClusterName + ServiceName (ECS); TargetGroup + LoadBalancer

Reading a CloudWatch metrics graph

Interpreting the graph:

```
1 aws cloudwatch get-metric-statistics \  
2   --namespace AWS/ECS \  
3   --metric-name CPUUtilization \  
4   --dimensions Name=ClusterName,Value=my-cluster \  
5                   Name=ServiceName,Value=backend-service \  
6   --start-time $(date -u -d '1 hour ago' +%FT%TZ) \  
7   --end-time   $(date -u +%FT%TZ) \  
8   --period 300 \  
9   --statistics Average
```

- Average CPU consistently >80% → service under stress; consider scaling out
- RunningTaskCount drops suddenly → tasks crashing (check logs)
- TargetResponseTime rising while CPU is low → downstream dependency (database) is the bottleneck



CloudWatch Alarms

- An alarm watches **one metric** over a rolling evaluation window
- Three states: **OK**, **ALARM**, **INSUFFICIENT_DATA**
- Transitions to ALARM trigger **actions** (SNS notification, Auto Scaling policy)

Alarm states

State	Meaning
OK	Metric is within the threshold for all evaluation periods
ALARM	Metric has breached the threshold for the required number of consecutive periods
INSUFFICIENT_DATA	Not enough data points yet – alarm just created, or metric has gaps

Why `INSUFFICIENT_DATA` matters:

- A newly deployed service has no metric history – the alarm starts here
- A stopped service (zero running tasks) produces no CPU data – alarm stays here
- Do not treat `INSUFFICIENT_DATA` as OK: it may mean the service is **down**



Create a CloudWatch Alarm

The alarm fires after **2 consecutive 5-minute periods** when the average CPU > 80%:

```
1 aws cloudwatch put-metric-alarm \  
2   --alarm-name backend-cpu-high \  
3   --namespace AWS/ECS \  
4   --metric-name CPUUtilization \  
5   --dimensions Name=ClusterName,Value=my-cluster \  
6                 Name=ServiceName,Value=backend-service \  
7   --statistic Average \  
8   --period 300 \  
9   --evaluation-periods 2 \  
10  --threshold 80 \  
11  --comparison-operator GreaterThanThreshold \  
12  --alarm-actions                                     ↵  
    ↵ arn:aws:sns:us-east-1:123456789012:ops-notifications \  
13  --ok-actions                                         ↵  
    ↵ arn:aws:sns:us-east-1:123456789012:ops-notifications
```



CloudWatch Notification

Notification flow:

- 1 Metric breach
- 2 CloudWatch Alarm
- 3 SNS topic
- 4 mail / PagerDuty / Lambda

Create CloudWatch email notification

Create topic and subscribe an email address to it:

```
1 aws sns create-topic --name ops-notifications
2 aws sns subscribe \
3   --topic-arn <ops-notifications-arn> \
4   --protocol email \
5   --notification-endpoint john@example.com
```

- The SNS ARN is used in `--alarm-actions` and `--ok-actions`
- Subscription must be **confirmed** by the recipient before emails are delivered



Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer
- 3 IAM Roles for ECS Tasks
- 4 CloudWatch for ECS
- 5 ECS Service Auto Scaling**
- 6 Closing

Auto Scaling concept

- ECS Service Auto Scaling adjusts the **desired task count** of a service
- Distinct from EC2 Auto Scaling (which adjusts instance count)
- Uses **Application Auto Scaling** as the underlying API

Two policy types:

Policy	Behaviour
Step scaling	Add/remove a fixed number of tasks on a CloudWatch Alarm
Target tracking	Adjust task count to keep a metric at a target value

Target tracking is the recommended default — simpler to configure, self-calibrating.

Registering a scalable target

Before creating policies, register the ECS service as a scalable target:

```
1 aws application-autoscaling register-scalable-target \  
2   --service-namespace ecs \  
3   --resource-id service/my-cluster/backend-service \  
4   --scalable-dimension ecs:service:DesiredCount \  
5   --min-capacity 2 \  
6   --max-capacity 8
```

- `--min-capacity 2` — **never** scale below 2 tasks (availability: one task can crash and service continues)
- `--max-capacity 8` — cost guardrail; prevents runaway scaling from a traffic spike or a bug
- The ECS service must already exist before registering it as a scalable target



Target-tracking scaling policy I

- Target: keep average CPU at **75 %**
- Scale-out cooldown: **60 s** — add tasks quickly
- Scale-in cooldown: **300 s** — wait 5 min before removing tasks

```
1 cat > policy.json << 'EOF'
2 {
3   "TargetValue": 75.0,
4   "PredefinedMetricSpecification": {
5     "PredefinedMetricType": "ECSServiceAverageCPUUtilization"
6   },
7   "ScaleOutCooldown": 60,
8   "ScaleInCooldown": 300
9 }
10 EOF
```



Target-tracking scaling policy II

Use **cooldown** periods to prevent rapid oscillation (thrashing) of scaling actions in response to burst spikes



Target-tracking scaling policy III

Create the application autoscaling policy:

```
1 aws application-autoscaling put-scaling-policy \  
2   --policy-name backend-cpu-tracking \  
3   --service-namespace ecs \  
4   --resource-id service/my-cluster/backend-service \  
5   --scalable-dimension ecs:service:DesiredCount \  
6   --policy-type TargetTrackingScaling \  
7   --target-tracking-scaling-policy-configuration file://policy.json
```

Scaling-out and scaling-in

Scale out - adding tasks:

- 1 Auto Scaling increases `DesiredCount` (e.g., 2 → 3)
- 2 ECS launches 2 new Fargate tasks — each gets an ENI and a private IP
- 3 New task IPs are registered in the ALB target group
- 4 ALB begins routing traffic to new tasks once health checks pass (~30–60s)

Scale in - removing tasks:

- 1 Auto Scaling decreases `DesiredCount` (e.g., 3 → 2)
- 2 ECS selects 2 tasks for termination
- 3 ALB deregisters the task IPs (connection draining — in-flight requests finish)
- 4 After deregistration delay (default: 300s), tasks are stopped

Control loop

Event	Metric, CloudWatch reaction	Auto Scaling operation
Load spike (sustained 60s)	CPU crosses 80%, Alarm: ALARM	scale-out policy fires
New tasks healthy	CPU drops to 55%, Alarm: OK	scale-in cooldown started
Cooldown expires (300s)	CPU stable at 50%	scale-in policy fires

The same metrics introduced in slide 34 (**CPUUtilization**, **RequestCountPerTarget**) are the direct inputs to both: alarms and auto scaling policies



Agenda

- 1 Networking Architecture for a Two-Tier ECS Deployment
- 2 Application Load Balancer
- 3 IAM Roles for ECS Tasks
- 4 CloudWatch for ECS
- 5 ECS Service Auto Scaling
- 6 Closing**



Before you go

On a piece of paper (or in your notes), write:

- 1 One thing from today you are **confident** about
- 2 One thing you are **still unsure** about — bring it to the lab

Exit ticket: *Your Spring Boot backend is logging to `stdout`. Where does that output end up, and which IAM role must have permission to write it there?*

Before you go

On a piece of paper (or in your notes), write:

- 1 One thing from today you are **confident** about
- 2 One thing you are **still unsure** about — bring it to the lab

Exit ticket: *Your Spring Boot backend is logging to `stdout`. Where does that output end up, and which IAM role must have permission to write it there?*

- Output → `awslogs` driver → CloudWatch Logs → log group `/ecs/<cluster>/<service>` → log stream per task
- The **task execution role** must have `logs:CreateLogStream` and `logs:PutLogEvents` — **not** the task role



References

- [1] Amazon Web Services, “Amazon elastic container service developer guide,” 2026. <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/> (accessed May 13, 2026).
- [2] Amazon Web Services, “Amazon virtual private cloud user guide,” 2026. <https://docs.aws.amazon.com/vpc/latest/userguide/> (accessed May 17, 2026).
- [3] Amazon Web Services, “Elastic load balancing user guide,” 2026. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/> (accessed May 17, 2026).
- [4] Amazon Web Services, “Amazon CloudWatch user guide,” 2026. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/> (accessed May 17, 2026).
- [5] Amazon Web Services, “Application auto scaling user guide,” 2026. <https://docs.aws.amazon.com/autoscaling/application/userguide/> (accessed May 17, 2026).



Wrocław
University
of Science
and Technology

Thank you for your attention

- See you next week